Software Design Document

Matrix Medical Network - UKG to BILL.com Integration

Version: 3.0 **Date:** March 25, 2026 **Document Status:** Complete

---

Table of Contents

1. Executive Summary
2. Multi-Repository Overview
3. System Overview
4. Architecture Overview
5. Module 1: Spend & Expense (S&E)
6. Module 2: Accounts Payable (AP)
7. Module 3: Motus Driver Sync
8. Module 4: TravelPerk SCIM Sync
9. Common Components & Patterns
10. Data Dictionary & DDL Scripts
11. API Endpoints Reference
12. Workflows & Process Flows
13. Security Considerations
14. Deployment Guide
15. Credentials & Secrets Inventory
16. 3rd Party Integrations Summary
17. Security Risk Assessment

---

1. Executive Summary

1.1 Purpose

This document provides a comprehensive software design specification for the Matrix Medical Network UKG to BILL.com Integration. The solution consists of a Python-based ETL (Extract, Transform, Load) pipeline that synchronizes data from **UKG Pro** (Ultimate Kronos Group workforce management system) to **BILL.com** for two distinct use cases:

1. **Spend & Expense (S&E)** - Employee provisioning for expense tracking and corporate card management
2. **Accounts Payable (AP)** - Vendor, bill, and payment management for accounts payable operations

1.2 Scope

The integration suite handles:

- Employee data extraction from UKG Pro REST APIs
- Data transformation and field mapping for BILL.com
- **S&E Module:** User provisioning (create/update/retire) in BILL.com Spend & Expense
- **AP Module:** Vendor management, bill/invoice creation, and payment processing
- Batch processing with parallel execution
- Rate limiting (60 calls/minute per BILL.com API limits)
- Error handling, retry logic, and logging
- Email notifications on completion/failure
- Correlation IDs for distributed tracing
- Docker containerization for deployment

1.3 Integration Suite Scope

This document covers the following integrations within the Matrix Medical Network integration suite:

Integration	Repository	Status	Description
BILL.com S&E	vai-matrix-ukg-bill-final	Active	Employee provisioning for Spend & Expense

Integration	Repository	Status	Description
BILL.com AP	vai-matrix-ukg-bill-final	Active	Vendor, bill, and payment management
Motus	vai-matrix-ukg-motus-final	Active	Driver reimbursement synchronization
TravelPerk	vai-matrix-ukg-travelperk-final	Active	SCIM 2.0 user provisioning

Out of Scope:

- BILL.com Accounts Receivable (AR)

1.4 Target Audience

- Software Engineers
- DevOps Engineers
- System Administrators
- Integration Architects
- QA Engineers

---

2. Multi-Repository Overview

2.1 Integration Suite Architecture

The Matrix Medical Network integration suite consists of **four independent repositories** that synchronize employee data from UKG Pro to various target systems:

```
                    ┌──────────────────────────────────┐
                    │            UKG Pro               │
                    │       (Source of Truth)          │
                    │                                  │
                    │  • Employee Data                 │
                    │  • Employment Details            │
                    │  • Person Details                │
                    │  • Cost Center/Dept Info         │
                    └──────────────────────────────────┘
                                   │
                ┌──────────────────┼──────────────────┐
                │                  │                  │
                ▼                  ▼                  ▼
┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐
│       BILL.com       │  │        Motus         │  │      TravelPerk      │
│     Integration      │  │      Integration     │  │      Integration     │
├──────────────────────┤  ├──────────────────────┤  ├──────────────────────┤
│ vai-matrix-ukg-bill-final │ vai-matrix-ukg-motus-final│ │vai-matrix-ukg-travelperk- │
│                      │  │                      │  │        final        │
│ • S&E User Sync      │  │ • Driver Sync        │  │ • SCIM User Sync     │
│ • AP Management      │  │ • FAVR/CPM Programs  │  │ • Supervisor Mgmt    │
│ • Browser Automation │  │ • Wave Deployments   │  │ • Two-Phase Batch    │
└──────────────────────┘  └──────────────────────┘  └──────────────────────┘
            │                      │                      │
            ▼                      ▼                      ▼
       BILL.com API           Motus REST API         TravelPerk SCIM 2.0
```

2.2 Repository Details

Repository	Location	Purpose	Architecture	LOC
vai-matrix-ukg-bill-final	`/reukgtomotussourcecode/`	UKG → BILL.com S&E + AP	Clean Architecture	~4,780
vai-matrix-ukg-bill-final	`/MatrixMedical/` (Legacy)	UKG → BILL.com S&E	Monolithic Scripts	~2,700
vai-matrix-ukg-motus-final	`/reukgtomotussourcecode/`	UKG → Motus Drivers	Monolithic Scripts	~800
vai-matrix-ukg-travelperk-final	`/reukgtomotussourcecode/`	UKG → TravelPerk SCIM	Monolithic Scripts	~1,048

2.3 Repository Independence

Key Point: All repositories are **completely independent** with no shared code or dependencies between them.

Characteristic	Description
No Shared Code	Each repository contains all required modules
Independent Containers	Each has its own Dockerfile and can be deployed separately
Shared Source System	All use UKG Pro with identical authentication patterns
Separate Target Systems	Each integrates with a different external API
Independent Execution	Can run simultaneously without conflicts
No Inter-Repository Communication	No message queues or shared state

2.4 Common UKG Authentication Pattern

All repositories authenticate to UKG Pro using the same pattern:

```
# Authentication headers (identical across all repos)
headers = {
    "Authorization": f"Basic {base64(username:password)}",
    "US-Customer-Api-Key": "{api_key}",
    "Accept": "application/json",
    "Content-Type": "application/json"
}
```

2.5 Deployment Overview

Each repository can be deployed independently as a Docker container:

```
# BILL.com Integration
docker run --rm --env-file matrix-ukg-bill.env matrix-ukg-bill:latest

# Motus Integration
docker run --rm --env-file matrix-ukg-motus.env matrix-ukg-motus:latest

# TravelPerk Integration
docker run --rm --env-file matrix-ukg-travelperk.env matrix-ukg-travelperk:latest
```

---

3. System Overview

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          UKG PRO (Source System)                         │
│                     https://service4.ultipro.com                         │
│  ┌────────────────┐  ┌────────────────┐  ┌──────────────────────────┐   │
│  │ Personnel API  │  │ Employment API │  │ Configuration API        │   │
│  │ - person-details│ │ - employment   │  │ - locations              │   │
│  │ - supervisor   │  │ - employee-    │  │                          │   │
│  │                │  │   employment   │  │                          │   │
│  └────────┬───────┘  └────────┬───────┘  └────────────┬─────────────┘   │
└───────────│──────────────────│──────────────────────│──────────────────┘
            │                  │                      │
            └──────────────────┼──────────────────────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     INTEGRATION LAYER (Python 3.11)                      │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                    vai-matrix-ukg-bill-final                     │   │
│  │                                                                  │   │
│  │  ┌──────────────────────────┐    ┌──────────────────────────┐   │   │
│  │  │   SPEND & EXPENSE (S&E)  │    │   ACCOUNTS PAYABLE (AP)  │   │   │
│  │  │                          │    │                          │   │   │
│  │  │  - build-bill-entity.py  │    │  - build-bill-vendor.py  │   │   │
│  │  │  - upsert-bill-entity.py │    │  - upsert-bill-vendor.py │   │   │
│  │  │  - run-bill-batch.py     │    │  - build-bill-invoice.py │   │   │
│  │  │  - orchestrate.py        │    │  - upsert-bill-invoice.py│   │   │
│  │  │  - scraping/             │    │  - process-bill-payment.py│  │   │
│  │  │                          │    │  - run-ap-batch.py       │   │   │
│  │  └──────────────────────────┘    └──────────────────────────┘   │   │
│  │                                                                  │   │
│  │  ┌──────────────────────────────────────────────────────────┐   │   │
│  │  │                       COMMON MODULES                     │   │   │
│  │  │  - secrets_manager.py  - rate_limiter.py   - correlation.py  │   │
│  │  │  - notifications.py    - metrics.py        - redaction.py    │   │
│  │  │  - validators.py       - report_generator.py                │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  │                                                                  │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
                               │
                               │ API Token
                               │ apiToken: xxx
                               ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                              BILL.com                                    │
│                                                                          │
│  ┌──────────────────────────┐    ┌──────────────────────────────┐       │
│  │  Spend & Expense API     │    │  Accounts Payable API        │       │
│  │  /v3/spend/              │    │  /v3/                        │       │
│  │                          │    │                              │       │
│  │  Endpoints:              │    │  Endpoints:                  │       │
│  │  - GET/POST /users       │    │  - GET/POST /vendors         │       │
│  │  - PATCH/DELETE /users/{id}│  │  - PATCH /vendors/{id}       │       │
│  │  - GET /users/current    │    │  - GET/POST /bills           │       │
│  │                          │    │  - PATCH /bills/{id}         │       │
│  │  Entity: User            │    │  - POST /payments            │       │
│  │  - email, firstName      │    │  - POST /payments/bulk       │       │
│  │  - lastName, role        │    │  - POST /bills/record-payment │      │
│  │  - retired, externalId   │    │                              │       │
│  │                          │    │  Entities: Vendor, Bill,     │       │
```

```
│  │                           │    │   Payment                       │    │
│  └───────────────────────────┘    └─────────────────────────────────┘    │
│                                                                          │
│  Base URL (Staging): https://gateway.stage.bill.com/connect/v3           │
│  Base URL (Production): https://gateway.bill.com/connect/v3              │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

2.2 Technology Stack

Component	Technology	Version
Language	Python	3.11
HTTP Client	requests	>=2.31.0
Configuration	python-dotenv	>=1.0.0
Browser Automation	Playwright	>=1.40.0
Container Runtime	Docker	Python 3.11-slim
Concurrency	ThreadPoolExecutor	stdlib

2.3 Repository Structure

```
reukgtomotussourcecode/
├── vai-matrix-ukg-bill-final/          # BILL.com Integration
│   │
│   ├── [SPEND & EXPENSE MODULE]
│   ├── build-bill-entity.py            # S&E: User payload builder
│   ├── upsert-bill-entity.py           # S&E: User API upserter
│   ├── run-bill-batch.py               # S&E: Batch orchestrator
│   ├── orchestrate_people_import.py    # S&E: End-to-end orchestrator
│   ├── scraping/                       # S&E: Playwright UI automation
│   │   └── run-bill-user-scrape.py     # CSV import via browser
│   │
│   ├── [ACCOUNTS PAYABLE MODULE]
│   ├── build-bill-vendor.py            # AP: Vendor payload builder
│   ├── upsert-bill-vendor.py           # AP: Vendor API upserter
│   ├── build-bill-invoice.py           # AP: Bill/invoice builder
│   ├── upsert-bill-invoice.py          # AP: Bill API upserter
│   ├── process-bill-payment.py         # AP: Payment processor
│   ├── run-ap-batch.py                 # AP: Batch orchestrator
│   │
│   ├── [CONFIGURATION]
│   ├── matrix-ukg-bill.env             # Environment configuration
│   ├── Dockerfile                      # Container definition
│   ├── requirements.txt                # Dependencies
│   ├── docs/                           # Field mapping documentation
│   └── data/                           # Output directory
│
├── common/                             # Shared modules
│   ├── __init__.py                     # Module exports
│   ├── secrets_manager.py              # Secrets management (SOW 2.6)
│   ├── rate_limiter.py                 # Rate limiting (SOW 5.1, 5.2)
│   ├── correlation.py                  # Correlation IDs (SOW 7.2)
│   ├── notifications.py                # Email alerts (SOW 4.6)
│   ├── metrics.py                      # Metrics collection (SOW 4.7)
│   ├── report_generator.py             # Run reports (SOW 7.3)
│   ├── redaction.py                    # PII redaction (SOW 7.4, 9.4)
│   └── validators.py                   # Data validation (SOW 3.6, 3.7)
│
├── scripts/                            # Utility scripts
│   ├── security_check.py               # Security validation (SOW 9.6)
│   ├── batch_wrapper.py                # SOW-compliant batch runner
│   ├── validation_runner.py            # Automated validation
│   └── load_test.py                    # Load testing
│
├── templates/                          # Email and report templates
├── tests/                              # Test suite
├── .gitignore                          # Security-focused ignores
└── Software_Design_Document.md         # This document
```

---

4. Architecture Overview

4.1 Design Patterns

The project follows consistent architectural patterns across both modules:

4.1.1 Three-Layer Architecture

```
┌──────────────────────────────────┐
│          ORCHESTRATION LAYER        │
│    (run-*-batch.py, run-ap-batch.py)│
│  - CLI argument parsing             │
│  - Batch processing coordination    │
│  - ThreadPoolExecutor management    │
│  - Progress reporting               │
│  - RunContext integration           │
└──────────────────────────────────┘
                   │
                   │
┌──────────────────▼──────────────────┐
│           BUILDER LAYER             │
│ (build-bill-entity.py,              │
│  build-bill-vendor.py, etc.)        │
│  - UKG API data extraction          │
│  - Field mapping & transformation   │
│  - Payload construction             │
│  - Validation                       │
└──────────────────────────────────┘
                   │
                   │
┌──────────────────▼──────────────────┐
│           UPSERTER LAYER            │
│ (upsert-bill-entity.py,             │
│  upsert-bill-vendor.py, etc.)       │
│  - BILL.com API authentication      │
│  - Rate limiting                    │
│  - CRUD operations                  │
│  - Retry logic & error handling     │
└──────────────────────────────────┘
```

4.1.2 Component Interaction

```
# Typical execution flow (S&E Module)
def main():
    # 1. Orchestration Layer
    with RunContext(project="bill", company_id=args.company_id) as ctx:
        args = parse_cli()
        employees = fetch_employees_from_ukg(args.company_id)

        # 2. Builder Layer (per employee)
        for employee in employees:
            payload = build_bill_entity(employee.number, employee.company_id)

            # 3. Upserter Layer
            rate_limiter.acquire()  # 60 calls/min
            result = upsert_bill_entity(payload, dry_run=args.dry_run)
            ctx.stats['created'] += 1

    # 4. Generate reports and send notifications
    report_generator.generate_run_report(ctx.to_dict())
    notifier.send_run_summary(ctx.to_dict())
```

4.2 Authentication Mechanisms

System	Authentication Type	Implementation
UKG Pro	Basic Auth + API Key	`Authorization: Basic base64(user:pass)` + `US-CUSTOMER-API-KEY` header
BILL.com S&E API	API Token	`apiToken: {token}` header
BILL.com AP API	API Token	`apiToken: {token}` header
BILL.com Web (Scraping)	Form Login	Browser automation with credentials

4.3 Error Handling Strategy

```
┌──────────────────────────────────────────────────────────┐
│                    ERROR HANDLING                        │
├──────────────────────────────────────────────────────────┤
│ 2xx Success     → Continue, record success               │
│ 401/403 Auth    → Log error, alert, fail batch           │
│ 404 Not Found   → Create new (INSERT)                    │
│ 409 Conflict    → Search by alternate key, then UPDATE   │
│ 429 Rate Limit  → Wait and retry (handled by limiter)    │
│ 4xx Client      → Log error, skip record, continue       │
│ 5xx Server      → Exponential backoff, max 2 retries     │
└──────────────────────────────────────────────────────────┘
```

4.4 Rate Limiting

BILL.com API has a rate limit of **60 calls per minute**. The integration uses a token bucket rate limiter:

```
# common/rate_limiter.py
BILL_RATE_LIMITER = TokenBucketRateLimiter(calls_per_minute=60)

def bill_api_call(...):
    BILL_RATE_LIMITER.acquire()  # Blocks if rate exceeded
    return requests.post(...)
```

---

5. Module 1: Spend & Expense (S&E)

6.1 Module Overview

Purpose: Synchronize employee data from UKG Pro to BILL.com Spend & Expense platform for expense tracking and corporate card management.

Directory: `vai-matrix-ukg-bill-final/`

6.2 Component Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    UKG-BILL S&E INTEGRATION                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌─────────────────┐    ┌─────────────────┐    ┌──────────────┐ │
│  │ run-bill-       │──▶│ build-bill-     │──▶│ upsert-bill-│ │
│  │ batch.py        │    │ entity.py       │    │ entity.py   │ │
│  └────────┬────────┘    └─────────────────┘    └──────┬───────┘ │
│           │                                           │        │
│           │              ┌─────────────────┐          │        │
│           └──────────────▶│ orchestrate_    │          │        │
│                           │ people_import.py│◀─────────┘        │
│                           └────────┬────────┘                  │
│                                    │                           │
│                           ┌────────▼────────┐                  │
│                           │ scraping/       │                  │
│                           │ run-bill-user-  │                  │
│                           │ scrape.py       │                  │
│                           └─────────────────┘                  │
└──────────────────────────────────────────────────────────────┘
```

6.3 File Descriptions

File	Lines	Purpose
`build-bill-entity.py`	~259	Fetches UKG data and builds BILL.com user payload
`upsert-bill-entity.py`	~387	BILL API operations (GET/POST/PATCH/DELETE users)
`run-bill-batch.py`	~505	Batch orchestrator with threading and CSV export
`orchestrate_people_import.py`	~95	End-to-end orchestrator (batch + scraping)
`scraping/run-bill-user-scrape.py`	~890	Playwright browser automation for CSV import

5.4 Data Flow

```
UKG Pro
    │
    ├──▶ GET /personnel/v1/employee-employment-details
    │        ↓
    │    {employeeNumber, employeeId, companyId, employeeTypeCode,
    │     primaryProjectCode, terminationDate}
    │
    ├──▶ GET /personnel/v1/employment-details
    │        ↓
    │    {employeeNumber, employeeStatusCode, supervisorEmployeeId}
    │
    └──▶ GET /personnel/v1/person-details
             ↓
         {firstName, lastName, emailAddress, addressLine1,
          addressCity, addressState, addressZipCode, phone}
             │
             ▼
    ┌──────────────────────────────────────────┐
    │         build_bill_entity()              │
    │  Transform UKG data → BILL.com payload   │
    │  + Email validation                      │
    │  + State code validation                 │
    └──────────────────────────────────────────┘
                        │
                        ▼
    ┌──────────────────────────────────────────┐
    │           BILL.com User Payload          │
    │  {email, firstName, lastName, role,      │
    │   retired, externalId, contact{...}}     │
    └──────────────────────────────────────────┘
                        │
                        ▼
    ┌──────────────────────────────────────────┐
    │         upsert_user_payload()            │
    │  Rate limit → Check exists → POST/PATCH  │
    └──────────────────────────────────────────┘
                        │
                        ▼
                 BILL.com S&E API
```

5.5 Field Mapping

UKG Source	UKG Field	BILL Field	Notes
person-details	emailAddress	email	Required, unique identifier
person-details	firstName	firstName	Required
person-details	lastName	lastName	Required
employment-details	employeeStatusCode	retired	A=active(false), else true
employee-employment-details	employeeNumber	externalId	Custom tracking
employee-employment-details	primaryProjectCode	custom.projectCode	Stored in metadata
person-details	addressLine1	contact.address1	Optional
person-details	addressCity	contact.city	Optional
person-details	addressState	contact.state	Validated against US states

UKG Source	UKG Field	BILL Field	Notes
person-details	addressZipCode	contact.postalCode	Optional
person-details	homePhone/mobilePhone	contact.phone	Normalized XXX-XXX-XXXX

5.6 BILL.com S&E API Endpoints

Base URL: `/v3/spend/`

Endpoint	Method	Purpose
`/users`	GET	List/search users with filters
`/users`	POST	Create new user
`/users/{uuid}`	GET	Get single user by UUID
`/users/{uuid}`	PATCH	Update user fields
`/users/{uuid}`	DELETE	Retire/delete user
`/users/current`	GET	Get authenticated user

5.7 User Roles

Role	Description
ADMIN	Full administrative access
AUDITOR	Read-only access to all data
BOOKKEEPER	Financial data access
MEMBER	Standard user access
NO_ACCESS	Disabled user

5.8 Known Limitations

1. **`retired` field not updatable via PATCH** - PATCH returns 200 but doesn't update; use DELETE instead
2. **`retired` cannot be set on creation** - Always creates with `retired: false`
3. **Custom fields not supported** - `customFields` in PATCH doesn't save
4. **No manager/supervisor via API** - Requires UI automation for manager assignment

6.9 CLI Usage

```
# Single employee build
python build-bill-entity.py 000479 J9A6Y

# Batch processing with filters
python run-bill-batch.py \
  --company-id J9A6Y \
  --states FL,MS,NJ \
  --employee-type-codes FTC,HRC,TMC \
  --workers 12 \
  --limit 10 \
  --dry-run

# Docker execution
docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  --company-id J9A6Y
```

---

6. Module 2: Accounts Payable (AP)

6.1 Module Overview

Purpose: Manage vendors, bills/invoices, and payments in BILL.com Accounts Payable for procurement and payment workflows.

Directory: `vai-matrix-ukg-bill-final/`

6.2 Component Architecture

```
┌──────────────────────────────────────────────────────────┐
│                   UKG-BILL AP INTEGRATION                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │                   run-ap-batch.py                 │    │
│  │             (Orchestration Layer)                 │    │
│  └──────────────────────┬───────────────────────────┘    │
│                         │                                 │
│          ┌──────────────┼──────────────┐                  │
│          │              │              │                  │
│          ▼              ▼              ▼                  │
│  ┌────────────┐  ┌────────────┐  ┌────────────────┐      │
│  │ VENDORS    │  │ BILLS      │  │ PAYMENTS       │      │
│  │            │  │            │  │                │      │
│  │ build-bill-│  │ build-bill-│  │ process-bill-  │      │
│  │ vendor.py  │  │ invoice.py │  │ payment.py     │      │
│  │     │      │  │     │      │  │                │      │
│  │     ▼      │  │     ▼      │  │                │      │
│  │ upsert-bill-│ │ upsert-bill-│ │                │      │
│  │ vendor.py  │  │ invoice.py │  │                │      │
│  └────────────┘  └────────────┘  └────────────────┘      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

6.3 File Descriptions

File	Purpose
`build-bill-vendor.py`	Builds vendor payload from source data
`upsert-bill-vendor.py`	Creates/updates vendors in BILL.com
`build-bill-invoice.py`	Builds bill/invoice payload
`upsert-bill-invoice.py`	Creates/updates bills in BILL.com
`process-bill-payment.py`	Processes vendor payments
`run-ap-batch.py`	Orchestrates AP operations

6.4 Vendor Management

6.4.1 Vendor Data Flow

```
Source Data (CSV/API)
    |
    ▼
┌──────────────────────────────────────────┐
│         build_vendor_payload()           │
│  - Extract vendor information            │
│  - Map to BILL.com schema                │
│  - Validate required fields              │
└──────────────────┬───────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────────┐
│           Vendor Payload                 │
│  {name, shortName, email, address,       │
│   paymentTermDays, paymentMethod}        │
└──────────────────┬───────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────────┐
│       upsert_vendor()                    │
│  GET /vendors (search by name/email)     │
│  → Not Found: POST /vendors (create)     │
│  → Found: PATCH /vendors/{id} (update)   │
└──────────────────┬───────────────────────┘
                   |
                   ▼
            BILL.com AP API
```

6.4.2 Vendor Payload Structure

```
{
  "name": "Vendor Name",
  "shortName": "VEND",
  "email": "vendor@example.com",
  "address": {
    "line1": "123 Main St",
    "city": "San Francisco",
    "state": "CA",
    "zip": "94105",
    "country": "US"
  },
  "paymentTermDays": 30,
  "paymentMethod": "CHECK"
}
```

6.4.3 Payment Methods

Method	Description
ACH	Electronic bank transfer
CHECK	Physical check
WIRE	Wire transfer
CARD_ACCOUNT	Virtual card payment

6.5 Bill/Invoice Management

6.5.1 Bill Data Flow

```
Source Data
    |
    ▼
┌──────────────────────────────────────┐
│        build_invoice_payload()       │
│  - Extract invoice information       │
│  - Resolve vendor ID                 │
│  - Map line items                    │
│  - Calculate due date                │
└──────────────────┬───────────────────┘
                   |
                   ▼
┌──────────────────────────────────────┐
│          Bill Payload                │
│  {vendorId, invoice{number, date},   │
│   dueDate, billLineItems[]}          │
└──────────────────┬───────────────────┘
                   |
                   ▼
┌──────────────────────────────────────┐
│       upsert_bill()                  │
│  GET /bills (search by invoice number)│
│  → Not Found: POST /bills (create)   │
│  → Found: PATCH /bills/{id} (update) │
└──────────────────┬───────────────────┘
                   |
                   ▼
           BILL.com AP API
```

6.5.2 Bill Payload Structure

```
{
  "vendorId": "vendor-uuid",
  "invoice": {
    "number": "INV-001",
    "date": "2026-03-22"
  },
  "dueDate": "2026-04-22",
  "billLineItems": [
    {
      "amount": 1000.00,
      "description": "Services rendered"
    }
  ]
}
```

6.6 Payment Processing

6.6.1 Payment Data Flow

```
Bills with status=APPROVED
    |
    ▼
┌──────────────────────────────────────────┐
│        get_payment_options()             │
│  GET /v3/payments/options                │
│  - Get available funding accounts        │
│  - Get supported payment methods         │
└──────────────────┬───────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────────┐
│        process_payment()                 │
│  POST /v3/payments (single)              │
│  - OR -                                  │
│  POST /v3/payments/bulk (multiple)       │
│  - MFA required for payments             │
└──────────────────┬───────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────────┐
│        record_external_payment()         │
│  POST /v3/bills/record-payment           │
│  - Record payments made outside BILL.com │
└──────────────────────────────────────────┘
```

6.6.2 Payment Payload Structure

```
{
  "billId": "bill-uuid",
  "processDate": "2026-03-22",
  "amount": 1000.00,
  "fundingAccount": {
    "type": "BANK_ACCOUNT",
    "id": "account-uuid"
  }
}
```

6.7 BILL.com AP API Endpoints

Base URL: `/v3/`

Endpoint	Method	Purpose
`/vendors`	GET	List vendors
`/vendors`	POST	Create vendor
`/vendors/{id}`	GET	Get vendor
`/vendors/{id}`	PATCH	Update vendor
`/bills`	GET	List bills
`/bills`	POST	Create bill
`/bills/{billId}`	GET	Get bill
`/bills/{billId}`	PATCH	Update bill
`/payments`	POST	Create single payment
`/payments/bulk`	POST	Create bulk payments
`/payments/options`	GET	Get payment options
`/bills/record-payment`	POST	Record external payment

6.8 MFA Requirements

IMPORTANT: Payment operations (`POST /v3/payments`, `POST /v3/payments/bulk`) require an MFA-trusted API session. The integration must:

1. Obtain MFA-trusted session before payment processing

2. Handle MFA challenges programmatically or via user interaction
3. Cache MFA-trusted session for subsequent calls

6.9 CLI Usage

```
# Vendor sync only
python run-ap-batch.py --company-id J9A6Y --vendors

# Bill/invoice sync only
python run-ap-batch.py --company-id J9A6Y --bills

# Payment processing only
python run-ap-batch.py --company-id J9A6Y --payments

# Full AP sync (vendors → bills → payments)
python run-ap-batch.py --company-id J9A6Y --all

# Dry-run mode
python run-ap-batch.py --company-id J9A6Y --all --dry-run

# Docker execution
docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  ap --company-id J9A6Y --all
```

---

7. Module 3: Motus Driver Sync

7.1 Module Overview

Purpose: Synchronize employee data from UKG Pro to Motus driver reimbursement platform for mileage tracking and vehicle expense management.

Repository: `vai-matrix-ukg-motus-final`

Location: `/Users/dhavalrajendradesai/Projects/MatrixMedical/reukgtomotussourcecode/vai-matrix-ukg-motus-final`

7.2 Component Architecture

```
┌──────────────────────────────────────────────────────────┐
│                   UKG-MOTUS INTEGRATION                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌────────────────┐    ┌────────────────┐    ┌──────────┐ │
│  │ run-motus-     │──►│ build-motus-   │──►│ upsert-  │ │
│  │ batch.py       │    │ driver.py      │    │ motus-   │ │
│  │ (Orchestrator) │    │ (Builder)      │    │ driver.py│ │
│  └────────────────┘    └────────────────┘    └──────────┘ │
│                                                  │       │
│                        ┌────────────────┐        │       │
│                        │ motus-get-     │◄───────┘       │
│                        │ token.py       │                │
│                        │ (JWT Auth)     │                │
│                        └────────────────┘                │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

7.3 File Descriptions

File	Lines	Purpose

File	Lines	Purpose
`run-motus-batch.py`	~250	Batch orchestrator with CLI parsing and ThreadPoolExecutor
`build-motus-driver.py`	~300	Fetches UKG data and builds Motus driver payload
`upsert-motus-driver.py`	~260	Motus API operations (GET/POST/PUT drivers)
`motus-get-token.py`	~150	JWT token management for Motus API

7.4 Entity Relationship Diagram

```
┌──────────────────────────────────────────────────────────┐
│                      UKG Employee                        │
├──────────────────────────────────────────────────────────┤
│ employeeNumber (PK)                                      │
│ firstName, lastName                                      │
│ workEmail                                                │
│ primaryWorkPhone                                         │
│ hireDate                                                 │
│ terminationDate                                          │
│ employeeStatus                                           │
│ supervisorName, supervisorEmail                          │
│ costCenterDescription                                    │
│ primaryProjectCode                                       │
└──────────────────────────┬───────────────────────────────┘
                           │
                           │ Transforms to
                           ▼
┌──────────────────────────────────────────────────────────┐
│                      Motus Driver                        │
├──────────────────────────────────────────────────────────┤
│ id (PK) - Motus assigned                                 │
│ clientEmployeeId1 ← employeeNumber                       │
│ firstName ← firstName                                    │
│ lastName ← lastName                                      │
│ email ← workEmail                                        │
│ phoneNumber ← primaryWorkPhone (normalized)              │
│ hireDate (MM/DD/YYYY format)                             │
│ terminationDate (MM/DD/YYYY format)                      │
│ programId ← costCenterDescription (FAVR/CPM mapping)     │
│ managerEmail ← supervisorEmail                           │
│ customFields (division, costCenter, etc.)                │
│ status (ACTIVE, TERMINATED)                              │
└──────────────────────────────────────────────────────────┘
```

7.5 Field Mapping

UKG Field	Motus Field	Transformation
employeeNumber	clientEmployeeId1	Direct mapping
firstName	firstName	Direct mapping
lastName	lastName	Direct mapping
workEmail	email	Direct mapping
primaryWorkPhone	phoneNumber	Normalize to XXX-XXX-XXXX
hireDate	hireDate	ISO 8601 → MM/DD/YYYY
terminationDate	terminationDate	ISO 8601 → MM/DD/YYYY
employeeStatus	status	A → ACTIVE, else TERMINATED
costCenterDescription	programId	Map to FAVR/CPM program ID
supervisorEmail	managerEmail	Direct mapping
primaryProjectCode	customFields.projectCode	Store in custom fields

7.6 Program ID Mapping

Motus uses program IDs to assign drivers to specific reimbursement programs:

Cost Center Pattern	Program Type	Program ID

Cost Center Pattern	Program Type	Program ID
Contains "FAVR"	Fixed & Variable Rate	`favr_program_id`
Contains "CPM"	Cents Per Mile	`cpm_program_id`
Default	CPM	`cpm_program_id`

7.7 Motus API Endpoints

Base URL: Configured via `MOTUS_API_BASE` environment variable

Authentication: JWT Bearer Token

Method	Endpoint	Description	Request	Response
POST	`/oauth/token`	Get JWT token	`{grant_type, client_id, client_secret}`	`{access_token, token_type, expires_in}`
GET	`/drivers`	List drivers	Query: `clientEmployeeId1={id}`	`[{driver objects}]`
GET	`/drivers/{id}`	Get driver by ID	-	`{driver object}`
POST	`/drivers`	Create driver	Driver payload	`{id, status: "created"}`
PUT	`/drivers/{id}`	Update driver	Driver payload	`{id, status: "updated"}`

7.8 Sample Payloads

Create Driver Request

```
{
  "clientEmployeeId1": "027603",
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@matrixmedical.com",
  "phoneNumber": "555-123-4567",
  "hireDate": "01/15/2023",
  "programId": "12345",
  "managerEmail": "manager@matrixmedical.com",
  "status": "ACTIVE",
  "customFields": {
    "division": "Northeast",
    "costCenter": "711"
  }
}
```

Create Driver Response

```
{
  "id": "drv_abc123def456",
  "clientEmployeeId1": "027603",
  "status": "ACTIVE",
  "createdAt": "2026-03-25T10:30:00Z"
}
```

7.9 Wave-Based Deployment

The Motus integration supports phased rollouts using "waves":

```
# Deploy wave 1 (pilot users)
python run-motus-batch.py --company-id J9A6Y --wave 1

# Deploy wave 2 (department rollout)
python run-motus-batch.py --company-id J9A6Y --wave 2
```

7.10 CLI Usage

```
# Build single driver payload
python build-motus-driver.py 000479 J9A6Y

# Batch processing
python run-motus-batch.py \
  --company-id J9A6Y \
  --states FL,MS,NJ \
  --workers 12 \
  --dry-run

# Docker execution
docker run --rm \
  --env-file matrix-ukg-motus.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-motus:latest \
  --company-id J9A6Y
```

7.11 Environment Variables

Variable	Required	Description
MOTUS_API_BASE	Yes	Motus API base URL
MOTUS_CLIENT_ID	Yes	OAuth client ID
MOTUS_CLIENT_SECRET	Yes	OAuth client secret
MOTUS_FAVR_PROGRAM_ID	Yes	FAVR program identifier
MOTUS_CPM_PROGRAM_ID	Yes	CPM program identifier
UKG_*	Yes	UKG credentials (shared pattern)

---

8. Module 4: TravelPerk SCIM Sync

8.1 Module Overview

Purpose: Synchronize employee data from UKG Pro to TravelPerk using SCIM 2.0 protocol for travel booking and expense management.

Repository: `vai-matrix-ukg-travelperk-final`

Location: `/Users/dhavalrajendradesai/Projects/MatrixMedical/reukgtomotussourcecode/vai-matrix-ukg-travelperk-final`

8.2 Component Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                   UKG-TRAVELPERK INTEGRATION                 │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌────────────────┐    ┌────────────────┐    ┌────────────┐ │
│  │ run-travelperk- │──►│ build-travelperk│──►│ upsert-    │ │
│  │ batch.py       │    │ -user.py       │    │ travelperk- │ │
│  │ (Orchestrator) │    │ (SCIM Builder) │    │ user.py    │ │
│  └────────────────┘    └────────────────┘    └────────────┘ │
│          │                                                   │
│          │ Two-Phase Processing                              │
│          │                                                   │
│          ├──► Phase 1: Create/Update users (no managers)     │
│          │                                                   │
│          └──► Phase 2: Update manager references             │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

8.3 File Descriptions

File	Lines	Purpose

File	Lines	Purpose
`run-travelperk-batch.py`	~408	Batch orchestrator with two-phase processing
`build-travelperk-user.py`	~242	Builds SCIM 2.0 compliant user payloads
`upsert-travelperk-user.py`	~398	TravelPerk SCIM API operations

8.4 Entity Relationship Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                      UKG Employee                            │
├──────────────────────────────────────────────────────────────┤
│ employeeNumber (PK)                                          │
│ firstName, lastName                                          │
│ workEmail                                                    │
│ employeeStatus                                               │
│ supervisorId                                                 │
│ jobTitle                                                     │
│ costCenterDescription                                        │
│ departmentCode                                               │
└──────────────────────────────┬───────────────────────────────┘
                               │
                               │ Transforms to
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                  SCIM 2.0 User Resource                      │
├──────────────────────────────────────────────────────────────┤
│ schemas: ["urn:ietf:params:scim:schemas:core:2.0:User",      │
│           "urn:ietf:params:scim:schemas:extension:enterprise │
│            :2.0:User"]                                       │
│                                                              │
│ id (PK) ← TravelPerk assigned UUID                           │
│ externalId ← employeeNumber                                  │
│ userName ← workEmail                                         │
│ name.givenName ← firstName                                   │
│ name.familyName ← lastName                                   │
│ emails[].value ← workEmail                                   │
│ emails[].primary ← true                                      │
│ active ← (employeeStatus == 'A')                             │
│ title ← jobTitle                                             │
│                                                              │
│ Enterprise Extension:                                        │
│ urn:...:enterprise:2.0:User                                  │
│   .manager.value ← supervisor's TravelPerk ID                │
│   .manager.displayName ← supervisor's name                   │
│   .costCenter ← costCenterDescription                        │
│   .department ← departmentCode                               │
└──────────────────────────────────────────────────────────────┘
```

8.5 Two-Phase Processing

TravelPerk requires manager references to point to existing users. The integration handles this with two-phase processing:

```
┌──────────────────────────────────────────────────────────────┐
│                    TWO-PHASE PROCESSING                      │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  PHASE 1: Create/Update Users (No Managers)                  │
│  ──────────────────────────────────────────                  │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────┐   │
│  │ Employee A   │    │ Employee B   │    │ Employee C   │   │
│  │ (Manager of  │    │ (Manager of  │    │ (Reports to  │   │
│  │  B and C)    │    │  none)       │    │  A)          │   │
│  └──────┬───────┘    └──────┬───────┘    └──────┬───────┘   │
│         │                   │                   │           │
│         ▼                   ▼                   ▼           │
│  ┌──────────────────────────────────────────────────┐       │
│  │ Create SCIM users WITHOUT manager field          │       │
│  │ Store mapping: employeeNumber → TravelPerk ID    │       │
│  └──────────────────────────────────────────────────┘       │
│                                                              │
│  PHASE 2: Update Manager References                          │
│  ──────────────────────────────────                          │
│  ┌──────────────────────────────────────────────────┐       │
│  │ For each employee with supervisor:               │       │
│  │   1. Lookup supervisor's TravelPerk ID           │       │
│  │   2. PATCH user with manager.value = supervisor_id  │       │
│  └──────────────────────────────────────────────────┘       │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

8.6 Field Mapping

UKG Field	SCIM Field	Transformation
employeeNumber	externalId	Direct mapping
workEmail	userName	Direct mapping
firstName	name.givenName	Direct mapping
lastName	name.familyName	Direct mapping
workEmail	emails[0].value	Primary email
employeeStatus	active	A → true, else false
jobTitle	title	Direct mapping
supervisorId	manager.value	Resolved in Phase 2
costCenterDescription	costCenter (enterprise)	Enterprise extension
departmentCode	department (enterprise)	Enterprise extension

8.7 TravelPerk SCIM 2.0 Endpoints

Base URL: `https://api.travelperk.com/scim/v2`

Authentication: API Key Header (`Api-Key: {token}`)

Method	Endpoint	Description
GET	`/Users`	List all users
GET	`/Users?filter=externalId eq "{id}"`	Find user by external ID
GET	`/Users/{id}`	Get user by TravelPerk ID
POST	`/Users`	Create new user
PUT	`/Users/{id}`	Full user update
PATCH	`/Users/{id}`	Partial update (used for manager)
DELETE	`/Users/{id}`	Delete user

8.8 Sample Payloads

Create User Request (Phase 1)

```
{
  "schemas": [
    "urn:ietf:params:scim:schemas:core:2.0:User",
    "urn:ietf:params:scim:schemas:extension:enterprise:2.0:User"
  ],
  "externalId": "027603",
  "userName": "john.doe@matrixmedical.com",
  "name": {
    "givenName": "John",
    "familyName": "Doe"
  },
  "emails": [
    {
      "value": "john.doe@matrixmedical.com",
      "type": "work",
      "primary": true
    }
  ],
  "active": true,
  "title": "Sales Representative",
  "urn:ietf:params:scim:schemas:extension:enterprise:2.0:User": {
    "costCenter": "711 - Northeast Sales",
    "department": "Sales"
  }
}
```

Create User Response

```
{
  "schemas": ["urn:ietf:params:scim:schemas:core:2.0:User"],
  "id": "tp_uuid_abc123",
  "externalId": "027603",
  "userName": "john.doe@matrixmedical.com",
  "meta": {
    "resourceType": "User",
    "created": "2026-03-25T10:30:00Z",
    "lastModified": "2026-03-25T10:30:00Z"
  }
}
```

Update Manager Request (Phase 2)

```
{
  "schemas": ["urn:ietf:params:scim:api:messages:2.0:PatchOp"],
  "Operations": [
    {
      "op": "replace",
      "path": "urn:ietf:params:scim:schemas:extension:enterprise:2.0:User:manager",
      "value": {
        "value": "tp_uuid_manager456",
        "displayName": "Jane Smith"
      }
    }
  ]
}
```

8.9 CLI Usage

```
# Build single user payload
python build-travelperk-user.py 000479 J9A6Y

# Batch processing (two-phase)
python run-travelperk-batch.py \
  --company-id J9A6Y \
  --states FL,MS,NJ \
  --employee-type-codes FTC,HRC \
  --workers 12 \
  --dry-run

# Docker execution
docker run --rm \
  --env-file matrix-ukg-travelperk.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-travelperk:latest \
  --company-id J9A6Y
```

8.10 Environment Variables

Variable	Required	Description
TRAVELPERK_API_BASE	Yes	TravelPerk SCIM API base URL
TRAVELPERK_API_KEY	Yes	TravelPerk API key
UKG_*	Yes	UKG credentials (shared pattern)

8.11 Known Limitations

1. **Manager Resolution Requires Two Passes** - Cannot set manager on creation if manager doesn't exist yet
2. **SCIM Pagination** - Large organizations may require pagination handling for user lists
3. **Rate Limits** - TravelPerk API has rate limits that require throttling

---

9. Common Components & Patterns

9.1 Common Modules

All modules are located in the `common/` directory:

Module	SOW Req	Purpose
`secrets_manager.py`	2.6	Secrets management (env, AWS, Vault)
`rate_limiter.py`	5.1, 5.2	Token bucket rate limiting
`correlation.py`	7.2	Correlation IDs and RunContext
`notifications.py`	4.6	Email notifications
`metrics.py`	4.7, 7.3	Metrics collection
`report_generator.py`	4.7, 7.3	Run summary reports
`redaction.py`	7.4, 9.4	PII/secrets redaction
`validators.py`	3.6, 3.7	Data validation

9.2 Secrets Management

```
from common import get_secrets_manager

secrets = get_secrets_manager()  # Auto-detects provider
api_token = secrets.get_secret("BILL_API_TOKEN")

# Supported providers:
# - EnvSecretsManager (development)
# - AWSSecretsManager (production)
# - VaultSecretsManager (alternative)
```

9.3 Rate Limiting

```
from common import get_rate_limiter

limiter = get_rate_limiter("bill")  # 60 calls/min

def make_api_call():
    limiter.acquire()  # Blocks if rate exceeded
    return requests.post(...)
```

9.4 Correlation IDs

```
from common import RunContext, get_logger

logger = get_logger(__name__)

with RunContext(project="bill", company_id="J9A6Y") as ctx:
    logger.info("Starting batch")  # Includes correlation ID
    # Process records...
    ctx.stats['created'] += 1
    ctx.record_error("EMP123", "Invalid email")
```

9.5 Notifications

```
from common import get_notifier

notifier = get_notifier()

# Send run summary
notifier.send_run_summary(run_data)

# Send critical alert
notifier.send_critical_alert(
    title="Batch Failed",
    error=exception,
    context={"correlation_id": ctx.correlation_id}
)
```

9.6 Environment Loading

```
def load_dotenv_simple(env_path: str = ".env") -> dict:
    """Simple .env parser without external dependencies"""
    result = {}
    if not os.path.exists(env_path):
        return result
    with open(env_path, "r") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            if "=" in line:
                key, _, value = line.partition("=")
                result[key.strip()] = value.strip()
    for k, v in result.items():
        os.environ.setdefault(k, v)
    return result
```

9.7 UKG Authentication Headers

```
def headers() -> dict:
    """Build UKG API request headers"""
    return {
        "Authorization": f"Basic {_get_token()}",
        "US-CUSTOMER-API-KEY": os.environ.get("UKG_CUSTOMER_API_KEY", ""),
        "Accept": "application/json",
        "Content-Type": "application/json"
    }

def _get_token() -> str:
    """Encode Basic Auth from environment"""
    b64 = os.environ.get("UKG_BASIC_B64", "")
    if b64:
        return b64
    user = os.environ.get("UKG_USERNAME", "")
    pwd = os.environ.get("UKG_PASSWORD", "")
    return base64.b64encode(f"{user}:{pwd}".encode()).decode()
```

9.8 Phone Normalization

```
def normalize_phone(val: Optional[str]) -> str:
    """Normalize phone to XXX-XXX-XXXX format"""
    if not val:
        return ""
    digits = re.sub(r"\D", "", val)
    if len(digits) == 10:
        return f"{digits[0:3]}-{digits[3:6]}-{digits[6:10]}"
    return val
```

9.9 Retry Logic

```
def retry_with_backoff(func, max_retries=2, base_delay=1.0):
    """Execute function with exponential backoff retry"""
    for attempt in range(max_retries + 1):
        try:
            response = func()
            if response.status_code < 500:
                return response
        except Exception as e:
            if attempt == max_retries:
                raise

        delay = base_delay * (2 ** attempt)
        time.sleep(delay)

    return response
```

9.10 Command-Line Interface Pattern

```
def parse_cli():
    parser = argparse.ArgumentParser()

    # Required
    parser.add_argument("--company-id", required=True, help="UKG company ID")

    # Filtering
    parser.add_argument("--states", help="Comma-separated US state codes")
    parser.add_argument("--employee-type-codes", help="Comma-separated type codes")
    parser.add_argument("--limit", type=int, default=0, help="Process N records")

    # Execution modes
    parser.add_argument("--dry-run", action="store_true", help="Validate only")
    parser.add_argument("--save-local", action="store_true", help="Save JSON files")

    # Performance
    parser.add_argument("--workers", type=int, default=12, help="Thread pool size")

    return parser.parse_args()
```

9.11 ThreadPoolExecutor Pattern

```
def process_batch(records, process_func, max_workers=12):
    """Process records in parallel with progress reporting"""
    results = {"saved": 0, "skipped": 0, "errors": 0}
    mapping = {}

    with ThreadPoolExecutor(max_workers=max_workers) as executor:
        futures = {
            executor.submit(process_func, record): record
            for record in records
        }

        for future in as_completed(futures):
            record = futures[future]
            try:
                result = future.result()
                if result.get("id"):
                    mapping[record["id"]] = result["id"]
                    results["saved"] += 1
                else:
                    results["skipped"] += 1
            except Exception as e:
                results["errors"] += 1
                logging.error(f"Error processing {record}: {e}")

    return results, mapping
```

---

10. Data Dictionary & DDL Scripts

10.1 Overview

This project does not use traditional relational databases. Data persistence is handled through:

- JSON files for payload storage and ID mappings
- CSV files for bulk import operations
- In-memory dictionaries for processing state

10.2 UKG Pro Data Models

10.2.1 Employment Details Entity

```
-- Conceptual DDL (not actual database)
CREATE TABLE ukg_employee_employment_details (
    employeeNumber      VARCHAR(20) PRIMARY KEY,
    employeeId          VARCHAR(50),       -- Internal UUID
    companyId           VARCHAR(20),       -- Company reference
    employeeTypeCode    VARCHAR(10),       -- FTC, HRC, TMC, etc.
    employeeStatusCode  CHAR(1),           -- A=Active, I=Inactive
    primaryProjectCode  VARCHAR(20),       -- Cost center
    primaryJobCode      VARCHAR(20),       -- Job classification
    locationCode        VARCHAR(20),       -- Work location
    terminationDate     DATE,              -- NULL if active
    originalHireDate    DATE,              -- First hire date
    lastHireDate        DATE               -- Most recent hire date
);
```

10.2.2 Person Details Entity

```
CREATE TABLE ukg_person_details (
    employeeId          VARCHAR(50) PRIMARY KEY,
    firstName           VARCHAR(100),
    lastName            VARCHAR(100),
    middleName          VARCHAR(100),
    emailAddress        VARCHAR(255) UNIQUE,
    addressLine1        VARCHAR(255),
    addressLine2        VARCHAR(255),
    addressCity         VARCHAR(100),
    addressState        CHAR(2),           -- US state code
    addressZipCode      VARCHAR(20),
    homePhone           VARCHAR(20),
    mobilePhone         VARCHAR(20)
);
```

10.3 BILL.com Data Models

10.3.1 S&E User Entity

```
CREATE TABLE bill_user (
    uuid            VARCHAR(50) PRIMARY KEY,   -- usr_xxxxx format
    id              VARCHAR(50),               -- Base64 encoded
    email           VARCHAR(255) UNIQUE NOT NULL,
    firstName       VARCHAR(100) NOT NULL,
    lastName        VARCHAR(100) NOT NULL,
    role            ENUM('ADMIN', 'AUDITOR', 'BOOKKEEPER', 'MEMBER', 'NO_ACCESS'),
    retired         BOOLEAN DEFAULT FALSE,
    externalId      VARCHAR(50),               -- UKG employeeNumber
    createdTime     TIMESTAMP
);
```

10.3.2 AP Vendor Entity

```sql
CREATE TABLE bill_vendor (
    id              VARCHAR(50) PRIMARY KEY,
    name            VARCHAR(255) NOT NULL,
    shortName       VARCHAR(50),
    email           VARCHAR(255),
    address_line1   VARCHAR(255),
    address_city    VARCHAR(100),
    address_state   CHAR(2),
    address_zip     VARCHAR(20),
    address_country CHAR(2) DEFAULT 'US',
    paymentTermDays INT DEFAULT 30,
    paymentMethod   ENUM('ACH', 'CHECK', 'WIRE', 'CARD_ACCOUNT'),
    createdTime     TIMESTAMP
);
```

10.3.3 AP Bill Entity

```sql
CREATE TABLE bill_bill (
    id              VARCHAR(50) PRIMARY KEY,
    vendorId        VARCHAR(50) NOT NULL,
    invoiceNumber   VARCHAR(100) NOT NULL,
    invoiceDate     DATE,
    dueDate         DATE,
    amount          DECIMAL(10,2),
    status          ENUM('DRAFT', 'PENDING', 'APPROVED', 'PAID'),
    createdTime     TIMESTAMP,
    FOREIGN KEY (vendorId) REFERENCES bill_vendor(id)
);

CREATE TABLE bill_line_item (
    id              VARCHAR(50) PRIMARY KEY,
    billId          VARCHAR(50) NOT NULL,
    amount          DECIMAL(10,2) NOT NULL,
    description     VARCHAR(500),
    FOREIGN KEY (billId) REFERENCES bill_bill(id)
);
```

10.4 Local Storage Schemas

10.4.1 Employee to BILL ID Mapping

```json
// File: data/batch/employee_to_bill_id_mapping.json
{
  "employeeNumber": "billUuid",
  "027603": "usr_abc123",
  "004295": "usr_def456"
}
```

10.4.2 S&E User Payload

```
// File: data/batch/bill_entity_{employeeNumber}.json
{
  "email": "john.doe@matrixmedical.com",
  "firstName": "John",
  "lastName": "Doe",
  "role": "MEMBER",
  "retired": false,
  "externalId": "027603",
  "_metadata": {
    "employeeNumber": "027603",
    "employeeID": "G4BVU1000030",
    "companyID": "J9A6Y",
    "primaryProjectCode": "711"
  }
}
```

10.4.3 AP Vendor Payload

```
// File: data/batch/vendor_{vendorId}.json
{
  "name": "ABC Supplies Inc",
  "shortName": "ABC",
  "email": "accounts@abc.com",
  "address": {
    "line1": "123 Main St",
    "city": "Tampa",
    "state": "FL",
    "zip": "33601",
    "country": "US"
  },
  "paymentTermDays": 30,
  "paymentMethod": "ACH"
}
```

10.5 CSV Export Format (S&E)

```
First name,Middle initial,Last name,Email address,Role,Physical Card Status,Membership Status,Date Added,Budget Count,Manager
John,,Doe,john.doe@matrixmedical.com,Member,,,,,"supervisor@matrixmedical.com"
Jane,M,Smith,jane.smith@matrixmedical.com,Member,,,,,
```

---

11. API Endpoints Reference

11.1 UKG Pro API

Base URL: `https://service4.ultipro.com`

Authentication:

```
Authorization: Basic {base64(username:password)}
US-CUSTOMER-API-KEY: {api_key}
Content-Type: application/json
Accept: application/json
```

Endpoint	Method	Parameters	Response
`/personnel/v1/employee-employment-details`	GET	`companyId`, `employeeNumber`, `per_Page`	Array of employment records
`/personnel/v1/employment-details`	GET	`employeeNumber`, `companyID`	Single employment record
`/personnel/v1/person-details`	GET	`employeeId`	Person contact details

Endpoint	Method	Parameters	Response
`/personnel/v1/employee-supervisor-details`	GET	`per_Page`	Array of supervisor mappings
`/configuration/v1/locations/{code}`	GET	-	Location details

11.2 BILL.com Spend & Expense API

Base URL (Staging): `https://gateway.stage.bill.com/connect/v3/spend` **Base URL (Production):** `https://gateway.bill.com/connect/v3/spend`

Authentication:

```
apiToken: {api_token}
Content-Type: application/json
Accept: application/json
```

Endpoint	Method	Body/Params	Response
`/users`	GET	`page`, `pageSize`, `email`, `search`	User list (paginated)
`/users`	POST	`{email, firstName, lastName, role}`	Created user
`/users/{uuid}`	GET	-	User object
`/users/{uuid}`	PATCH	`{firstName?, lastName?, role?}`	Updated user
`/users/{uuid}`	DELETE	-	Success response
`/users/current`	GET	-	Current auth user

11.3 BILL.com Accounts Payable API

Base URL (Staging): `https://gateway.stage.bill.com/connect/v3` **Base URL (Production):** `https://gateway.bill.com/connect/v3`

Authentication:

```
apiToken: {api_token}
Content-Type: application/json
Accept: application/json
```

Vendor Endpoints

Endpoint	Method	Body/Params	Response
`/vendors`	GET	`page`, `pageSize`, `search`	Vendor list
`/vendors`	POST	Vendor payload	Created vendor
`/vendors/{id}`	GET	-	Vendor object
`/vendors/{id}`	PATCH	Partial vendor payload	Updated vendor

Bill Endpoints

Endpoint	Method	Body/Params	Response
`/bills`	GET	`page`, `pageSize`, `status`, `vendorId`	Bill list
`/bills`	POST	Bill payload	Created bill
`/bills/{billId}`	GET	-	Bill object
`/bills/{billId}`	PATCH	Partial bill payload	Updated bill

Payment Endpoints

Endpoint	Method	Body/Params	Response	Notes
`/payments`	POST	Payment payload	Created payment	MFA required
`/payments/bulk`	POST	Array of payments	Bulk result	MFA required
`/payments/options`	GET	`billId`	Payment options	
`/bills/record-payment`	POST	External payment	Recorded payment	

---

12. Workflows & Process Flows

12.1 S&E Batch Processing Flow

```
┌──────────────────────────────────────────────────────────────────────┐
│                    S&E BATCH PROCESSING FLOW                         │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  START                                                               │
│    │                                                                 │
│    ▼                                                                 │
│  ┌──────────────────────┐                                            │
│  │ 1. Initialize RunContext│                                         │
│  │    - Generate corr ID   │                                         │
│  │    - Configure logging  │                                         │
│  │    - Setup rate limiter │                                         │
│  └──────────┬───────────────┘                                        │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────┐                                            │
│  │ 2. Fetch Employees      │                                         │
│  │    - GET employee-      │                                         │
│  │      employment-details │                                         │
│  │    - Filter by company  │                                         │
│  └──────────┬───────────────┘                                        │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────────────────────────────────┐                │
│  │ 3. Process Employees (ThreadPoolExecutor)         │                │
│  │    ┌──────────────────────────────────────────┐│                │
│  │    │ FOR EACH employee:                        │└                │
│  │    │   a. Rate limit check (60/min)            │                 │
│  │    │   b. Fetch person-details                 │                 │
│  │    │   c. Validate (email, state code)         │                 │
│  │    │   d. Build BILL payload                   │                 │
│  │    │   e. Upsert to BILL.com                   │                 │
│  │    │   f. Record metrics                       │                 │
│  │    │   g. Handle errors, continue batch        │                 │
│  │    └──────────────────────────────────────────┘│                │
│  └──────────┬──────────────────────────────────────┘                │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────┐                                            │
│  │ 4. Generate Reports     │                                         │
│  │    - JSON run summary   │                                         │
│  │    - HTML report        │                                         │
│  │    - Validation results │                                         │
│  └──────────┬───────────────┘                                        │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────┐                                            │
│  │ 5. Send Notifications   │                                         │
│  │    - Email run summary  │                                         │
│  │    - Critical alerts    │                                         │
│  └──────────┬───────────────┘                                        │
│             │                                                        │
│             ▼                                                        │
│  END                                                                 │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

12.2 AP Batch Processing Flow

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        AP BATCH PROCESSING FLOW                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  START                                                                   │
│    │                                                                     │
│    ▼                                                                     │
│  ┌──────────────────────┐                                                │
│  │ 1. Initialize RunContext│                                             │
│  │    - Generate corr ID   │                                             │
│  │    - Parse CLI (--mode) │                                             │
│  └──────────┬───────────────┘                                            │
│             │                                                            │
│     ┌───────┴──────┬──────────────┐                                      │
│     │              │              │                                      │
│     ▼              ▼              ▼                                      │
│  VENDORS          BILLS         PAYMENTS                                 │
│     │              │              │                                      │
│     ▼              │              │                                      │
│  ┌──────────────┐  │              │                                      │
│  │ 2a. Sync     │  │              │                                      │
│  │     Vendors  │  │              │                                      │
│  │  - Load source │  │              │                                      │
│  │  - Build payld │  │              │                                      │
│  │  - Upsert API  │  │              │                                      │
│  │  - Store IDs   │  │              │                                      │
│  └──────┬───────┘  │              │                                      │
│         │          │              │                                      │
│         └──────────┴──────────────┘                                      │
│                    │                                                     │
│                    ▼                                                     │
│  ┌──────────────────────────────────┐                                    │
│  │ 2b. Sync Bills                   │                                    │
│  │  - Load invoice data             │                                    │
│  │  - Resolve vendor IDs            │                                    │
│  │  - Build bill payloads           │                                    │
│  │  - Upsert to BILL.com            │                                    │
│  └──────────────┬───────────────────┘                                    │
│                 │                                                        │
│                 ▼                                                        │
│  ┌──────────────────────────────────┐                                    │
│  │ 2c. Process Payments             │                                    │
│  │  - Get approved bills            │                                    │
│  │  - MFA authentication            │                                    │
│  │  - Process payments              │                                    │
│  │  - Record external payments      │                                    │
│  └──────────────┬───────────────────┘                                    │
│                 │                                                        │
│                 ▼                                                        │
│  ┌──────────────────────┐                                                │
│  │ 3. Generate Reports  │                                                │
│  │ 4. Send Notifications │                                               │
│  └──────────┬───────────┘                                                │
│             │                                                            │
│             ▼                                                            │
│  END                                                                     │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

12.3 Upsert Decision Flow

```
┌──────────────────────────────────────────────────────────────────────┐
│                          UPSERT DECISION FLOW                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Payload Ready                                                       │
│    │                                                                 │
│    ▼                                                                 │
│  ┌──────────────────────┐                                            │
│  │ Rate Limit Check     │                                            │
│  │ (60 calls/min)       │──── Wait if exceeded                       │
│  └──────────┬───────────┘                                            │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────┐                                            │
│  │ Validate Required    │                                            │
│  │ Fields               │──── Invalid ──▶ SKIP (log error)           │
│  └──────────┬───────────┘                                            │
│             │ Valid                                                  │
│             ▼                                                        │
│  ┌──────────────────────┐                                            │
│  │ Check if Record Exists │                                          │
│  │ (GET by ID/email)    │                                            │
│  └──────────┬───────────┘                                            │
│             │                                                        │
│     ┌───────┴────────┐                                               │
│     │                │                                               │
│     ▼                ▼                                               │
│   Found            Not Found                                         │
│     │                │                                               │
│     ▼                ▼                                               │
│  ┌─────────┐       ┌─────────┐                                      │
│  │ UPDATE  │       │ INSERT  │                                      │
│  │ (PATCH) │       │ (POST)  │                                      │
│  └────┬────┘       └────┬────┘                                      │
│       │                 │                                            │
│       │           ┌─────┴─────┐                                      │
│       │           │           │                                      │
│       │         Success    409 Conflict                              │
│       │           │           │                                      │
│       │           │           ▼                                      │
│       │           │     ┌──────────────┐                             │
│       │           │     │ Search by    │                             │
│       │           │     │ alternate key │                            │
│       │           │     └──────┬───────┘                             │
│       │           │            │                                     │
│       │           │        ┌───┴───┐                                 │
│       │           │      Found    Not Found                          │
│       │           │        │        │                                │
│       │           │        ▼        ▼                                │
│       │           │     UPDATE     ERROR                             │
│       │           │                                                  │
│       ▼           ▼                                                  │
│  ┌──────────────────────┐                                            │
│  │ Return Result        │                                            │
│  │ {action, status, id} │                                            │
│  └──────────────────────┘                                            │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

12.4 Orchestrated S&E Import Flow

```
┌──────────────────────────────────────────────────────────────────────┐
│                    S&E ORCHESTRATED IMPORT                           │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  orchestrate_people_import.py                                        │
│    │                                                                 │
│    ▼                                                                 │
│  ┌──────────────────────┐                                            │
│  │ 1. Run Batch Process  │                                            │
│  │    python run-bill-   │                                            │
│  │    batch.py           │                                            │
│  │    --company-id X     │                                            │
│  │    --states Y         │                                            │
│  └──────────┬───────────┘                                            │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────┐                                            │
│  │ 2. Generate CSV Export │                                            │
│  │    data/people-       │                                            │
│  │    {timestamp}.csv    │                                            │
│  └──────────┬───────────┘                                            │
│             │                                                        │
│             ▼                                                        │
│  ┌──────────────────────────────────────────────────┐                │
│  │ 3. Run Playwright Scraper (for UI-only fields)    │               │
│  │    ┌──────────────────────────────────────────┐│                │
│  │    │ a. Launch browser                         │                │
│  │    │ b. Login to BILL.com                      │                │
│  │    │ c. Navigate to /people                    │                │
│  │    │ d. Click "Import People" button           │                │
│  │    │ e. Upload CSV file                        │                │
│  │    │ f. Wait for completion                    │                │
│  │    └──────────────────────────────────────────┘│                │
│  └──────────────────────────────────────────────────┘                │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

---

13. Security Considerations

13.1 Credential Management

Secret	Storage	Recommendation
UKG_USERNAME	.env file	Use secrets manager in production
UKG_PASSWORD	.env file	Use secrets manager in production
UKG_CUSTOMER_API_KEY	.env file	Use secrets manager in production
BILL_API_TOKEN	.env file	Use secrets manager in production
BILL_LOGIN_EMAIL	.env file	Use secrets manager in production
BILL_LOGIN_PASSWORD	.env file	Use secrets manager in production

13.2 Environment File Security

```
# .gitignore (required)
*.env
.env.*
data/batch/*.json
data/*.csv
```

13.3 Data Classification

Data Type	Classification	Handling
Employee Names	PII	Encrypted in transit (HTTPS)
Email Addresses	PII	Encrypted in transit (HTTPS)
Phone Numbers	PII	Encrypted in transit (HTTPS)
Home Addresses	PII	Encrypted in transit (HTTPS)
Employment Dates	Internal	Encrypted in transit (HTTPS)
Vendor Information	Business	Encrypted in transit (HTTPS)
Payment Details	Sensitive	Encrypted in transit, MFA required
Credentials	Secret	Never log, store securely

13.4 API Security

The integration uses:

- **HTTPS** for all API communications
- **Authentication headers** (not query parameters)
- **Rate limiting** to prevent API abuse
- **No credential logging** (passwords masked)
- **PII redaction** in logs

13.5 Docker Security

```
# Run as non-root user (SOW 9.5)
RUN useradd -m -u 1000 -s /bin/bash appuser && \
    chown -R appuser:appuser /app
USER appuser

# No secrets in image
# Pass via --env-file at runtime
```

---

14. Deployment Guide

14.1 Prerequisites

- Python 3.11+
- Docker (for containerized deployment)
- Network access to:
  - `service4.ultipro.com` (UKG Pro)
  - `gateway.bill.com` (BILL.com Production)
  - `gateway.stage.bill.com` (BILL.com Staging)

14.2 Local Development Setup

```
# Clone repository
cd reukgtomotussourcecode

# Setup BILL integration
cd vai-matrix-ukg-bill-final
python -m venv venv
source venv/bin/activate  # or venv\Scripts\activate on Windows
pip install -r requirements.txt
cp matrix-ukg-bill.env.example matrix-ukg-bill.env
# Edit matrix-ukg-bill.env with credentials

# Test with dry-run (S&E)
python run-bill-batch.py --company-id J9A6Y --limit 1 --dry-run

# Test with dry-run (AP)
python run-ap-batch.py --company-id J9A6Y --vendors --dry-run
```

14.3 Docker Build & Run

```
cd vai-matrix-ukg-bill-final

# Build
docker build -t matrix-ukg-bill:latest .

# Run S&E batch (dry-run)
docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  se --company-id J9A6Y --limit 10 --dry-run

# Run S&E batch (production)
docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  se --company-id J9A6Y --states FL,MS,NJ

# Run AP batch (dry-run)
docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  ap --company-id J9A6Y --all --dry-run
```

14.4 Environment Configuration Template

```
# UKG Configuration
UKG_BASE_URL=https://service4.ultipro.com
UKG_USERNAME=<username>
UKG_PASSWORD=<password>
UKG_CUSTOMER_API_KEY=<api_key>

# BILL.com Configuration
BILL_API_BASE=https://gateway.bill.com/connect/v3    # Production
# BILL_API_BASE=https://gateway.stage.bill.com/connect/v3  # Staging
BILL_API_TOKEN=<api_token>

# BILL.com Web Scraper (for CSV import)
BILL_LOGIN_EMAIL=<email>
BILL_LOGIN_PASSWORD=<password>
BILL_COMPANY_NAME=<company_name>

# Notifications
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=<user>
SMTP_PASSWORD=<password>
ALERT_RECIPIENTS=ops@matrixmedical.com

# Processing
WORKERS=12
DEBUG=0
```

14.5 Scheduled Execution (Cron)

```
# Daily S&E sync - 6:00 AM UTC
0 6 * * * cd /app && docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  se --company-id J9A6Y

# Daily AP vendor/bill sync - 7:00 AM UTC
0 7 * * * cd /app && docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  ap --company-id J9A6Y --vendors --bills

# Weekly AP payment processing - Sunday 8:00 AM UTC
0 8 * * 0 cd /app && docker run --rm \
  --env-file matrix-ukg-bill.env \
  -v "$(pwd)/data:/app/data" \
  matrix-ukg-bill:latest \
  ap --company-id J9A6Y --payments
```

14.6 Monitoring & Logging

Log Output Format:

```
[INFO] [abc123-def456] Starting S&E batch processing: company=J9A6Y, workers=12
[INFO] [abc123-def456] Fetched 1500 employees from UKG
[INFO] [abc123-def456] Filtered to 450 employees (states=FL,MS,NJ)
[DEBUG] [abc123-def456] Processing employee 027603...
[DEBUG] [abc123-def456] Rate limiter: 45/60 calls used
[INFO] [abc123-def456] 100/450 | saved=98 skipped=2 errors=0
[INFO] [abc123-def456] Done: total=450 | saved=445 | skipped=3 | errors=2
```

Output Files:

- `data/batch/employee_to_bill_id_mapping.json` - S&E ID mappings
- `data/batch/vendor_id_mapping.json` - AP vendor ID mappings
- `data/batch/bill_entity_*.json` - Individual S&E payloads (if --save-local)
- `data/people-{timestamp}.csv` - S&E CSV export
- `data/reports/run_{correlation_id}.json` - Run summary
- `data/reports/run_{correlation_id}.html` - HTML report

14.7 Production Deployment Checklist

```
## Pre-Deployment Security Checklist

### Credential Management
- [ ] All .env files removed from version control
- [ ] Credentials migrated to secrets manager
- [ ] UKG password rotated
- [ ] BILL API token rotated

### Configuration
- [ ] Production endpoints configured (not sandbox)
- [ ] DEBUG=0 in all environments
- [ ] File permissions secured (600)
- [ ] .gitignore updated

### Infrastructure
- [ ] Docker running as non-root user
- [ ] Network access verified to external services
- [ ] Log aggregation configured
- [ ] Monitoring alerts set up

### Validation
- [ ] Dry-run completed successfully
- [ ] S&E batch tested with limit
- [ ] AP vendor sync tested
- [ ] Rate limiter verified (no 429 errors)
- [ ] Email notifications working
```

---

15. Credentials & Secrets Inventory

15.1 Environment Variables Summary

Shared Credentials

Variable	Type	Risk Level	Purpose
`UKG_BASE_URL`	URL	LOW	UKG API endpoint
`UKG_USERNAME`	Username	HIGH	UKG Basic Auth username
`UKG_PASSWORD`	Password	**CRITICAL**	UKG Basic Auth password
`UKG_CUSTOMER_API_KEY`	API Key	HIGH	UKG API header authentication
`UKG_BASIC_B64`	Encoded	HIGH	Optional pre-encoded Basic Auth

BILL.com Specific

Variable	Type	Risk Level	Purpose
`BILL_API_BASE`	URL	LOW	BILL API endpoint
`BILL_API_TOKEN`	API Token	**CRITICAL**	BILL API authentication
`BILL_LOGIN_EMAIL`	Email	HIGH	Web scraper login email
`BILL_LOGIN_PASSWORD`	Password	**CRITICAL**	Web scraper login password
`BILL_COMPANY_NAME`	String	LOW	Company selection in UI

Notification Variables

Variable	Type	Risk Level	Purpose
`SMTP_HOST`	String	LOW	SMTP server hostname
`SMTP_PORT`	Integer	LOW	SMTP port (587)
`SMTP_USER`	String	MEDIUM	SMTP username
`SMTP_PASSWORD`	Password	HIGH	SMTP password
`ALERT_RECIPIENTS`	String	LOW	Email recipients

Operational Variables

Variable	Type	Default	Purpose
`COMPANY_ID`	String	J9A6Y	UKG Company identifier

Variable	Type	Default	Purpose
`WORKERS`	Integer	12	ThreadPool size
`STATES`	String	-	State filter (comma-separated)
`DRY_RUN`	Boolean	0	Validation only mode
`SAVE_LOCAL`	Boolean	0	Save JSON payloads locally
`DEBUG`	Boolean	0	Debug logging
`LIMIT`	Integer	0	Process N records (0=all)
`MAX_RETRIES`	Integer	2	API retry attempts
`OUT_DIR`	Path	data/batch	Output directory

15.2 Environment File Location

```
reukgtomotussourcecode/
└── vai-matrix-ukg-bill-final/
    └── matrix-ukg-bill.env        # All BILL.com credentials
```

15.3 Authentication Methods

Service	Method	Token Type	Lifetime	Refresh
UKG Pro	HTTP Basic Auth + Header	N/A	N/A	N/A
BILL.com API	API Token Header	Static	Long-lived	Manual rotation
BILL.com Web	Form login	Session	Session	Per login

15.4 Credential Usage in Code

File	Credentials Used	Authentication Type
`build-bill-entity.py`	UKG_*	Basic Auth
`upsert-bill-entity.py`	BILL_API_TOKEN	API Token Header
`run-bill-user-scrape.py`	BILL_LOGIN_*	Form Login
`build-bill-vendor.py`	UKG_*	Basic Auth
`upsert-bill-vendor.py`	BILL_API_TOKEN	API Token Header
`upsert-bill-invoice.py`	BILL_API_TOKEN	API Token Header
`process-bill-payment.py`	BILL_API_TOKEN	API Token Header + MFA

---

16. 3rd Party Integrations Summary

16.1 Integration Matrix

Service	Type	Auth Method	Timeout	Retry	Rate Limit
UKG Pro	REST API	Basic Auth + API Key	45s	5 attempts	N/A
BILL.com S&E	REST API	API Token Header	60s	2 retries	60/min
BILL.com AP	REST API	API Token Header	60s	2 retries	60/min
BILL.com Web	Browser	Form Login	N/A	N/A	N/A
Motus	REST API	JWT Bearer Token	60s	2 retries	N/A
TravelPerk	SCIM 2.0	API Key Header	60s	2 retries	N/A

16.2 Endpoint Details

UKG Pro (Source System)

Endpoint	Method	Purpose
`/personnel/v1/employee-employment-details`	GET	Bulk employee data
`/personnel/v1/employment-details`	GET	Individual employment
`/personnel/v1/person-details`	GET	Personal info
`/personnel/v1/employee-supervisor-details`	GET	Supervisor hierarchy
`/configuration/v1/locations/{code}`	GET	Location lookup

BILL.com S&E API

Endpoint	Method	Purpose
`/users`	GET/POST	List/create users
`/users/{uuid}`	GET/PATCH/DELETE	User CRUD

BILL.com AP API

Endpoint	Method	Purpose
`/vendors`	GET/POST	List/create vendors
`/vendors/{id}`	GET/PATCH	Vendor CRUD
`/bills`	GET/POST	List/create bills
`/bills/{id}`	GET/PATCH	Bill CRUD
`/payments`	POST	Create payment (MFA)
`/payments/bulk`	POST	Bulk payments (MFA)

Motus API

Endpoint	Method	Purpose
`/oauth/token`	POST	JWT authentication
`/drivers`	GET	List/search drivers
`/drivers`	POST	Create driver
`/drivers/{id}`	GET	Get driver
`/drivers/{id}`	PUT	Update driver

TravelPerk SCIM 2.0 API

Endpoint	Method	Purpose
`/scim/v2/Users`	GET	List users
`/scim/v2/Users?filter=externalId eq "{id}"`	GET	Find user by external ID
`/scim/v2/Users`	POST	Create user
`/scim/v2/Users/{id}`	PUT	Full user update
`/scim/v2/Users/{id}`	PATCH	Partial update (manager)
`/scim/v2/Users/{id}`	DELETE	Delete user

16.3 Data Flow Summary

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          DATA FLOW OVERVIEW                              │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  UKG Pro (Source for all integrations)                                   │
│    │                                                                     │
│    ├──▶ employee-employment-details ──┬──▶ BILL.com S&E                 │
│    │                                  │     └▶ User with role, email     │
│    │                                  │                                  │
│    ├──▶ employment-details ───────────┼──▶ Motus                        │
│    │                                  │     └▶ Driver with program       │
│    │                                  │                                  │
│    └──▶ person-details ───────────────┼──▶ TravelPerk                   │
│                                       │     └▶ SCIM User with manager    │
│                                       │                                  │
│  External Data Sources                │                                  │
│    │                                  │                                  │
│    ├──▶ Vendor data ──────────────────▶ BILL.com AP Vendors             │
│    │                                                                     │
│    ├──▶ Invoice data ─────────────────▶ BILL.com AP Bills               │
│    │                                                                     │
│    └──▶ Payment data ──────────────────▶ BILL.com AP Payments            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

16.4 Webhook & Callback Status

No webhooks or callbacks configured. All integrations are:

- Request-response (synchronous)
- Initiated by batch scripts
- No real-time event handling

---

17. Security Risk Assessment

17.1 Critical Vulnerabilities

Issue	Severity	Location	Impact	Recommendation
Hardcoded passwords in .env	CRITICAL	Project .env	Credential exposure	Use secrets manager
Exposed API tokens	CRITICAL	.env files	Unauthorized access	Move to vault
Sandbox endpoints	HIGH	BILL_API_BASE	Not production-ready	Update URLs
No credential rotation	MEDIUM	All projects	Long-lived secrets	Implement rotation

17.2 Required .gitignore Entries

```
# Environment files with credentials
*.env
.env.*
matrix-ukg-*.env

# Output data (may contain PII)
data/
data/batch/
data/batch/*.json
data/*.csv
data/reports/

# IDE and system files
.DS_Store
.idea/
__pycache__/
*.pyc
```

17.3 Production Security Recommendations

Immediate Actions (Before Production)

1. **Secrets Management**

```
# Move credentials to AWS Secrets Manager
docker run --rm \
  -e UKG_PASSWORD="$(aws secretsmanager get-secret-value --secret-id ukg-prod --query SecretString --output text)" \
  matrix-ukg-bill:latest
```

2. **Credential Rotation**

   - Generate new UKG password
   - Rotate BILL API token
   - Update all environment files

3. **Endpoint Updates**

```
# Update from staging to production
BILL_API_BASE=https://gateway.bill.com/connect/v3  # Remove .stage
```

Runtime Security

4. **File Permissions**

```
chmod 600 matrix-ukg-bill.env
```

5. **Docker Security**

- Run as non-root user (already implemented)
- No secrets in image
- Pass secrets via --env-file at runtime

17.4 Production Security Checklist

```
## Pre-Deployment Security Checklist

### Credential Management
- [ ] All .env files removed from version control
- [ ] Credentials migrated to secrets manager
- [ ] UKG password rotated
- [ ] BILL API token rotated

### Configuration
- [ ] Production endpoints configured (not sandbox)
- [ ] DEBUG=0 in all environments
- [ ] File permissions secured (600)
- [ ] .gitignore updated

### Infrastructure
- [ ] Docker running as non-root user
- [ ] Network segmentation configured
- [ ] Audit logging enabled
- [ ] Monitoring alerts configured

### Compliance
- [ ] PII handling documented
- [ ] Data retention policy defined
- [ ] Access control implemented
- [ ] Security review completed
```

Appendix A: Glossary

Term	Definition
AP	Accounts Payable - managing vendor payments
BILL.com	Cloud-based financial operations platform
ETL	Extract, Transform, Load - data integration pattern
MFA	Multi-Factor Authentication
PII	Personally Identifiable Information
S&E	Spend & Expense - corporate card and expense management
UKG Pro	Ultimate Kronos Group - workforce management platform
Upsert	Update if exists, Insert if not

Appendix B: Version History

Version	Date	Author	Changes
1.0	2026-03-19	Auto-generated	Initial document with 3 integrations
1.1	2026-03-19	Auto-generated	Added security sections
2.0	2026-03-22	Auto-generated	**Major Update**: Removed Motus and TravelPerk integrations. Focused on BILL.com only with S&E and AP modules. Added common modules documentation.
3.0	2026-03-25	Auto-generated	**Major Update**: Re-added Motus and TravelPerk integrations. Added multi-repository overview with relationship diagram. Added Module 3 (Motus Driver Sync) and Module 4 (TravelPerk SCIM Sync) with ERDs, field mappings, and API documentation. Updated 3rd Party Integrations Summary.

End of Software Design Document